

SOLAREXPRESS

SYSTEM ARCHITECTURE DOCUMENT

Version 1.1 · August 2026 · Status: Production Release Candidate
Blockchain Interplanetary Ticketing — Next.js · Express · MongoDB · Ethereum Sepolia

CONTENTS

- 1. Overview
- 2. System Components
- 3. High-Level Request Flow
- 4. Frontend Application
- 5. Backend Application & REST API
- 6. Data Model (MongoDB)
- 7. Authentication & Authorization
- 8. Ticket Minting Pipeline
- 9. Blockchain Integration
- 10. External Services & Dependencies
- 11. Configuration & Environment
- 12. Security Considerations
- 13. Deployment
- 14. Testing & Verification
- 15. Known Issues & Technical Debt
- 16. Revision History

1. OVERVIEW

SolarExpress is a blockchain-based interplanetary ticketing application. Users create an account, explore a three-dimensional model of the solar system, and purchase NFT boarding passes to 20 celestial destinations on the Ethereum Sepolia test network. Purchases are recorded both on-chain (as non-fungible token mint events) and off-chain (in MongoDB) so that users can view a unified ticket history.

The system is composed of a Next.js frontend, an Express/Mongoose REST backend, and a set of deployed smart contracts. MongoDB is the only database in use; the blockchain and IPFS serve auxiliary roles (ownership ledger and metadata storage, respectively).

2. SYSTEM COMPONENTS

Component	Technology	Responsibility
Frontend	Next.js 16.2.11 (App Router, Turbopack), React 19, Tailwind CSS v4, Three.js, Wagmi/Viem, TanStack Query	User interface: 3D solar-system exploration, booking flow, wallet connection, booking history, FAQ and About pages.
Backend	Express 4, Mongoose, JSON Web Tokens, bcryptjs, Helmet, CORS	REST API for authentication, bookings, notifications, wallet linkage, and Pinata uploads. CommonJS module format.
Database	MongoDB (Mongoose ODM)	Persists users, bookings, and notifications. The only database in the system.
Blockchain	Ethereum Sepolia testnet + IPFS (Pinata)	Mints NFT boarding passes (TicketNFT), records purchases (TicketSale/BookingHistory), and stores ticket artwork/metadata (IPFS).

3. HIGH-LEVEL REQUEST FLOW

```
Browser → /api/* → next.config.ts rewrite proxy → Express backend
                    (default target: http://localhost:4000, overridable via API_BACKEND_URL)
```

Exception: /api/nft/generate-metadata is a Next.js Route Handler, executed in the frontend runtime (not proxied to the backend).

Client authentication is carried in an httpOnly cookie (credentials: 'include').
A Bearer token header is accepted only as a fallback.

4. FRONTEND APPLICATION

4.1 PAGE ROUTES

Route	Purpose
/	Home — planet menu (sidebar), Three.js solar-system scene (desktop) or 2D explore list (mobile), destination/route panels, and the booking action.
/booking-history	Unified ticket history merging MongoDB bookings with on-chain TicketPurchased logs.
/faq	Frequently asked questions.
/about	Project information.
/signin	Sign in / sign up. Redirects authenticated users to the home page.

4.2 PROVIDER HIERARCHY (APP/LAYOUT.TSX)

```
Web3Provider (Wagmi) → AuthProvider → LoadingProvider → NotificationProvider → children
                                                                → MobileBottomNav
```

Routes are guarded client-side: unauthenticated visitors to / are redirected to /signin, and authenticated visitors to /signin are redirected to /.

4.3 KEY MODULES (SRC/)

Module	Role
lib/auth-context.tsx	Session state, signin/signup/logout, refreshUser; cookie-based API calls.
lib/notification-context.tsx	Polls /api/notifications and exposes unread notifications.
lib/loading-context.tsx	Global loading overlay via LoadingProvider.
lib/api.ts	authFetch helper plus syncBooking and fetchBookingHistory (TanStack Query hooks).
lib/contract.ts	Sepolia contract addresses and ABIs (TicketSale, TicketNFT, BookingHistory).
lib/nft-service.ts	Client-side metadata generation orchestration.
data/bodies.ts	Catalog of 20 celestial bodies plus moons (id, name, color, price, parent links).
data/travel.ts	Route definitions, launch times, terminals, and business-class pricing (×2.5 economy).
hooks/useBuyTicket.ts	Wallet write: mints via buyTicket with an explicit gas limit of 500,000.
hooks/useOnChainBookings.ts	Reads TicketPurchased events for the connected wallet.
components/scene/SolarSystem.tsx	Three.js renderer for the solar system.
components/panel/BookTicketButton.tsx	Entry point for the booking flow (wallet connect, class toggle, mint, sync).

5. BACKEND APPLICATION & REST API

5.1 SERVER SETUP (BACKEND/SERVER.JS)

The Express application enables Helmet security headers, cookie parsing, CORS with credentials, and JSON body parsing with a 512 KB limit. It trusts one proxy hop and listens on `PORT` (default 4000) after the database connection is established. A health endpoint is exposed at `GET /api/health`.

5.2 ENDPOINT REFERENCE

Endpoint	Method	Auth	Description
/api/auth/signup	POST	Public	Create account; sets session cookie; rate-limited (20 per 15 min).
/api/auth/login	POST	Public	Authenticate; sets session cookie; rate-limited.

/api/auth/logout	POST	Public	Clears session cookie.
/api/auth/me	GET	Required	Returns the current user profile.
/api/bookings	POST	Required	Create a booking (requires a real transaction hash).
/api/bookings/history	GET	Required	List the authenticated user's bookings.
/api/users/wallet	PATCH	Required	Link a wallet address to the account (no frontend caller yet).
/api/users/wallet	DELETE	Required	Unlink the wallet address (no frontend caller yet).
/api/notifications	GET	Required	List the user's notifications.
/api/notifications/:id/read	PATCH	Required	Mark a notification as read.
/api/notifications/read-all	PATCH	Required	Mark all notifications as read.
/api/notifications/:id/dismiss	PATCH	Required	Dismiss a promotion notification.
/api/notifications	POST	Required	Create a notification.
/api/pinata/upload-ticket	POST	Public	Upload NFT SVG + metadata to IPFS via Pinata.

5.3 ONBOARDING NOTIFICATIONS

On signup or login, the backend seeds a welcome notification and an active promotion notification. Both are created at most once per user (queried by type rather than by dismissed state), so dismissing the promotion does not cause it to reappear on subsequent logins.

6. DATA MODEL (MONGODB)

MongoDB is the sole database. All three collections are managed by Mongoose with timestamps enabled.

6.1 COLLECTION: USERS

Field	Type	Notes
email	String	Required, unique, lowercased; validated email format.
passwordHash	String	Required, bcrypt hash (cost 10), excluded from queries by default.
username	String	Optional.
walletAddress	String	Optional, lowercased; used by linkage and contract-backfill scripts.
profileImage	String	Optional.
role	String	Enum: user admin (default user).
tickets[]	Array	Sub-documents: destinationId, txHash, tokenId, purchasedAt.
favorites[]	Array(Number)	Destination IDs; currently no frontend caller.

6.2 COLLECTION: BOOKINGS

Field	Type	Notes
userId	ObjectId	Ref User, required.
walletAddress	String	Required, lowercased.
destinationId	Number	Required (1–20).
transactionHash	String	Required, unique, lowercased; guards against duplicate syncs.
tokenId	Number	Required, indexed.
pricePaid	String	Required; stored as decimal ETH (e.g. "0.004").
status	String	Enum: Confirmed Refunded Cancelled (default Confirmed).
bookingReference	String	Unique, sparse; format SX-XXXXXX.
departureDate / departureTime	String	Flight schedule.
travelClass	String	Enum: economy business first (default economy); normalized on write.
seatNumber	String	Row + letter.
availabilityStatus	String	Seat availability snapshot.
flightNumber / launchTerminal	String	Generated flight details.

6.3 COLLECTION: NOTIFICATIONS

Field	Type	Notes
userId	ObjectId	Ref User, required.
type	String	Enum: welcome promotion transaction.
message	String	Required.
read	Boolean	Default false.
dismissed	Boolean	Default false; promotion dismissal.
tokenId / txHash	Number / String	Transaction notifications.

7. AUTHENTICATION & AUTHORIZATION

- Passwords are hashed with **bcryptjs** (cost factor 10) before storage; the hash is never selected by default.
- On success the backend issues a **JSON Web Token** (7-day expiry, signed with JWT_SECRET) and stores it in an **httpOnly cookie** named `token` (secure + sameSite none in production, lax locally).
- The `requireAuth` middleware reads the cookie first and accepts a Bearer header as a fallback; it attaches the user object to the request.
- Authentication endpoints are rate-limited to 20 requests per 15 minutes per client.
- Frontend requests use `credentials: 'include'`; the dev proxy keeps the browser and API on the same origin.

8. TICKET MINTING PIPELINE

The booking flow executes in four ordered stages:

#	Stage	Details
1	Generate metadata	POST /api/nft/generate-metadata (Next.js route, maxDuration 90s). Predicts the next token ID by scanning recent TicketPurchased logs, renders an SVG boarding pass, and uploads the SVG plus JSON metadata to IPFS via the backend's Pinata endpoint.
2	Mint	Wallet calls <code>buyTicket(destinationId, tokenId, metadataURI)</code> on TicketSale using <code>useBuyTicket</code> with an explicit gas limit of 500,000 (the network-measured estimate for this contract exceeds 349k gas).
3	Persist booking	POST /api/bookings creates the MongoDB record. Requires walletAddress, destinationId, transactionHash, tokenId, and pricePaid; the hash must be unique. Missing flight fields are filled by the backend fallback generator; travelClass is normalized to the schema enum.
4	Side effects	A BookingHistory on-chain event, an in-app transaction notification, and a ticket entry appended to the user record.

8.1 BOOKING HISTORY VIEW

The booking-history page merges MongoDB bookings with live on-chain TicketPurchased logs (keyed by the connected wallet and transaction hash) so that tickets minted by other tools still appear. Status fallbacks use the exact documented case ("Confirmed").

9. BLOCKCHAIN INTEGRATION

9.1 DEPLOYED CONTRACTS (SEPOLIA)

Contract	Address	Role
TicketSale	0x8032...91dC	Primary entry point; buyTicket() mints a ticket NFT and emits TicketPurchased.
TicketNFT	0x6909...31b	ERC-721 contract; tokenURI resolves to Pinata metadata.
BookingHistory	0xC1D9...5EF	Event log of purchase history used by the frontend and backfill scripts.

9.2 ON-CHAIN EVENT

```
event TicketPurchased(uint256 indexed tokenId,
    address indexed buyer,
    uint256 indexed destinationId,
    uint256 pricePaid);
```

Business-class pricing is enforced by the contract's destinationPrice function and matches the UI's $\times 2.5$ rule for all 20 destinations.

10. EXTERNAL SERVICES & DEPENDENCIES

Service	Role	Notes
MongoDB Atlas	The only database	Accessed through Mongoose; hosts users, bookings, notifications.
Pinata (IPFS)	NFT metadata + artwork storage	tokenURI points at the returned IPFS CID.
Sepolia public RPCs	Blockchain read/write transport	publicnode.com, 1rpc.io, drpc.org, tenderly; fallback-ranked, no API key required.
Render	Backend hosting	Configured via API_BACKEND_URL / NEXT_PUBLIC_API_URL.
Vercel	Frontend hosting	Serves the Next.js build and rewrites /api/* to the backend.
MetaMask / WalletConnect	Wallet connection	Managed through Wagmi/Viem.

11. CONFIGURATION & ENVIRONMENT

11.1 FRONTEND (.ENV.LOCAL / .ENV.EXAMPLE)

Variable	Purpose
NEXT_PUBLIC_TICKET_SALE_ADDRESS	TicketSale contract address.
NEXT_PUBLIC_TICKET_NFT_ADDRESS	TicketNFT contract address.
NEXT_PUBLIC_BOOKING_HISTORY_ADDRESS	BookingHistory contract address.
NEXT_PUBLIC_SEPOLIA_RPC_URL	Preferred Sepolia RPC endpoint.
API_BACKEND_URL	Backend URL used by the Next.js /api rewrite proxy.

11.2 BACKEND (BACKEND/.ENV)

Variable	Purpose
MONGODB_URI	MongoDB connection string (Atlas).
JWT_SECRET	Secret used to sign session tokens.
PORT	Server port (default 4000).
CORS_ORIGIN	Allowed frontend origin for credentialed CORS.
NODE_ENV	development production.
TICKET_SALE_ADDRESS / SOLAREXPRESS_TICKET_ADDRESS / BOOKING_HISTORY_ADDRESS	Contract addresses used by backend utilities and scripts.

12. SECURITY CONSIDERATIONS

- httpOnly, secure, sameSite cookies for session transport; JWT signed with a server-side secret.
- Passwords hashed with bcrypt (cost 10); password hashes excluded from default queries and API responses.
- Helmet security headers, CORS restricted to CORS_ORIGIN with credentials, and a 512 KB JSON body limit.
- Rate limiting on authentication routes to mitigate credential-stuffing and brute-force attempts.
- Unique indexes on user email and booking transactionHash prevent duplicate accounts and booking records.
- No secrets are committed; all keys are supplied via environment variables and documented in .env.example.

13. DEPLOYMENT

- **Frontend (Vercel):** production build generated by `next build` (Turbopack). The build outputs 8 static pages plus one dynamic route (`/api/nft/generate-metadata`).
- **Backend (Render):** Express server connects to MongoDB Atlas on boot; CORS_ORIGIN and API_BACKEND_URL point at the deployed frontend and backend respectively.
- **Data:** MongoDB Atlas cluster; the connection is established before the HTTP listener starts.
- **One-time maintenance scripts:** `backend/scripts/backfillFromContract.js` replays TicketPurchased events into MongoDB, using real block timestamps for record dates.

14. TESTING & VERIFICATION

Check	Result
TypeScript type check (<code>tsc --noEmit</code>)	Passes with no errors.
ESLint (<code>eslint .</code>)	Passes with no errors.

Production build (<code>next build</code>)	Compiles; 8 static pages + 1 dynamic route.
Backend syntax (<code>node --check</code>)	Passes on controllers, utils, and scripts.

15. KNOWN ISSUES & TECHNICAL DEBT

Item	Impact	Status
linkWallet / unlinkWallet endpoints have no frontend caller; User.favorites unused	Wallet linkage and favorites are API-only; contract backfill cannot match buyers unless walletAddress is populated	Known; feature gap
Token-ID prediction is heuristic (scan of recent logs)	Concurrent mints can produce an off-by-one in the SVG/name embedded in metadata	Known limitation
No scheduled cleanup of old notifications	Notification collection grows unbounded	Known; acceptable for testnet
Metadata generation depends on public RPC availability	Slow or failing under RPC congestion; mitigated by fallback-ranked providers and a 90s route budget	Mitigated

16. REVISION HISTORY

Version	Date	Summary of Changes
1.0	Jul 2026	Initial architecture outline; gas-limit fix (500k) deployed.
1.1	Aug 2026	Detailed data model, full endpoint reference, environment tables, security section. Working-tree fixes documented: travelClass enum normalization, Pluto on mobile, bottom-sheet layout fix, backfill block timestamps, promotion-created-once, booking status case, dead-code removal.

Document status note: Section 1.1 changes are verified locally (type check, lint, and production build all pass) but are pending commit and deployment at the time of writing.
